Universidade Federal do Maranhão

Curso de Engenharia da Computação

PLANEJAMENTO DO PROJETO

Sistema Distribuído para Auxílio de Dados Hospitalares

Relatório de Implementação e Atualização do Cronograma: Marco 3

Curso: Engenharia da Computação

Disciplina: Sistemas Distribuídos

Professor(a): Luiz Henrique Neves Rodrigues

Grupo: Fernando da Silva Costa; Samara Santos Viegas; Eduardo dos Santos Oliveira; Herick Vinicius Pinheiro Da Conceição; Vyktor Gabryel Costa de Souza

14 de outubro de 2026

Sumário

1 Introdução e Visão Geral **1**

2 Diagrama de Gantt e Atualização do Cronograma **1**

3 Análise e Modelagem do Dataset Hospitalar **2**
3.1 Origem e Caracterização dos Dados 2
3.2 Mapeamento Semântico para a API 3

4 Recapitulação dos Marcos 1 e 2 **3**
4.1 Marco 1: Servidor Base e Ingestão de Dados 3
4.2 Marco 2: Cluster com Balanceador de Carga Round Robin 3
4.3 O Problema Identificado no Marco 2 (Motivação do Marco 3) 4

5 Marco 3 — Middleware MOM, Replicação e Escalabilidade **4**
5.1 Nova Topologia Integrada do Sistema Distribuído 5
5.2 Middleware Orientado a Mensagens (RabbitMQ) 5
5.3 Replicação Ativa Broadcast (Padrão Fanout) 6
5.4 Escalabilidade Horizontal Elástica (Scale-Out e Scale-In) 6
5.5 Visualização em Tempo Real e Acesso Remoto Seguro 7

6 Decisões de Engenharia e Desafios Superados no Marco 3 **7**

7 Validação e Resultados Experimentais **8**
7.1 Comprovação Experimental da Divisão de Carga 8

8 Roteiro Prático de Demonstração e Verificação (Marco 3) **8**
8.1 Inicialização Integrada do Cluster 8
8.2 Envio de Telemetria via Middleware AMQP 9
8.3 Demonstração da Escalabilidade Elástica 9
8.4 Evidência da Replicação Consistente nos 4 Bancos 9

9 Conclusão e Próximos Passos **10**

1 Introdução e Visão Geral

Em ambientes hospitalares críticos, a telemetria contínua de sinais vitais de pacientes gera um fluxo massivo de dados em tempo real. Se esse tráfego converge para um único servidor centralizado, surgem gargalos severos de processamento, alta latência nas consultas clínicas e, fundamentalmente, a vulnerabilidade crítica de ponto único de falha (*Single Point of Failure* – SPoF). A indisponibilidade de tal sistema acarreta riscos à assistência médica.

Com o objetivo de projetar, construir e validar uma solução robusta orientada aos conceitos de Sistemas Distribuídos, desenvolvemos o **Sistema Distribuído para Auxílio de Dados Hospitalares**. O projeto adota uma metodologia evolutiva estruturada em marcos de maturidade crescente:

- **Marco 1 — Simulação e Servidor Base (Concluído):** Ingestão de dados reais de monitoramento biomédico (dataset clínico do Kaggle com 60.000 registros), validação formal de tipos através de modelos Pydantic na API RESTful (FastAPI) e persistência relacional local (SQLite).
- **Marco 2 — Balanceamento de Carga e Múltiplas Instâncias (Concluído):** Implantação de um cluster de três instâncias de aplicação independentes (:8001, :8002 e :8003) coordenadas por um Balanceador de Carga Round Robin (:8000), garantindo transparência de acesso e divisão estocasticamente uniforme do tráfego de entrada.
- **Marco 3 — Middleware Orientado a Mensagens (MOM), Replicação Ativa e Escalabilidade Elástica (Escopo Deste Relatório):** Evolução arquitetural com a introdução do RabbitMQ (AMQP 5672) para desacoplamento espaço-temporal, replicação de dados em broadcast *Fanout* para sincronização de 100% dos registros em todos os nós (eliminando a inconsistência e erros 404 do Marco 2), mecanismo de escalabilidade horizontal elástica (*Scale-Out* e *Scale-In*) com 4º nó em *Warm Standby* (:8004) e interface gráfica de visualização com suporte a túnel público seguro e fallback inteligente.

O projeto é integralmente implementado em Python (versão 3.10+) com tipagem estática e arquitetura modular desacoplada.

2 Diagrama de Gantt e Atualização do Cronograma

A Figura 1 ilustra o cronograma do projeto atualizado na data de **14 de outubro de 2026**. As barras em **verde** representam as etapas previamente finalizadas e homologadas (Planejamento, Ambiente e os Marcos 1 e 2). A barra em **amarelo** evidencia o **Marco 3 (Middleware, Replicação e Escalabilidade)**, objeto da presente entrega e em fase de consolidação. As barras em **cinza** delineiam as etapas futuras previstas para o Marco 4, baterias de injeção de falhas e encerramento.

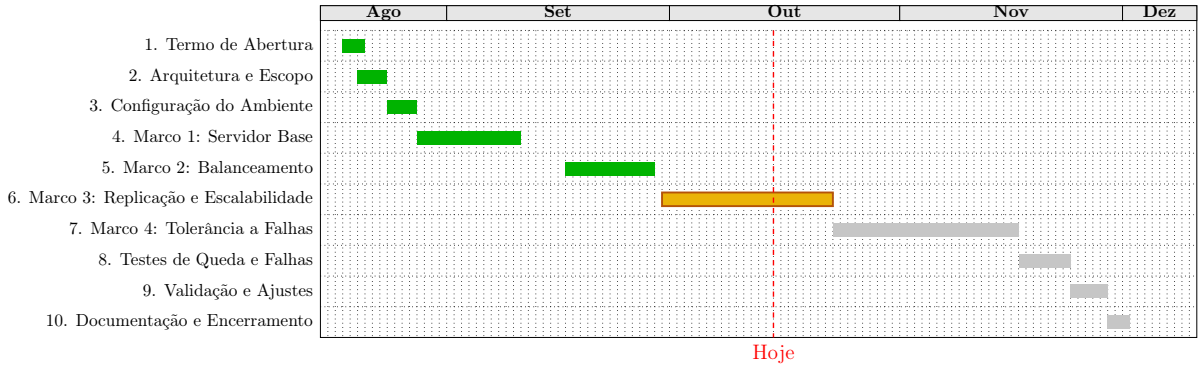


Figura 1: Diagrama de Gantt atualizado em 14/10/2026. Destaque em **amarelo** para a entrega do Marco 3.

Síntese da evolução cronológica:

- **Fases Concluídas (Verde):** Termo de abertura, levantamento de requisitos, arquitetura de rede, Marco 1 (API e SQLite) e Marco 2 (Load Balancer e cluster de 3 nós).
- **Fase Atual em Destaque (Amarelo): Marco 3**, contemplando a introdução do Middleware RabbitMQ, a replicação Fanout entre todos os nós, o 4º servidor reserva em *Warm Standby* e o suporte a acesso público via túnel reverso.
- **Fases Subsequentes (Cinza):** O Marco 4 focará no endurecimento da tolerância a falhas ativas, chaveamento automático de falhas (failover sem intervenção humana) e encerramento documental.

3 Análise e Modelagem do Dataset Hospitalar

3.1 Origem e Caracterização dos Dados

Os dados de telemetria derivam do dataset público do Kaggle:

`Synthetic_patient-HealthCare-Monitoring_dataset.csv`

A opção pelo uso de um repositório volumoso pré-existente viabiliza simulações fidedignas a cenários médicos reais. As principais características métricas são:

- **Volume:** 60.000 amostras clínicas ordenadas (60.001 linhas com o cabeçalho).
- **Integridade:** 0% de dados nulos ou registros ausentes em todas as variáveis.
- **Temporalidade:** O dataset não possui carimbo de tempo intrínseco. Assim, a camada de ingestão atribui o *timestamp* de alta precisão (ISO 8601) no momento da recepção em rede.

3.2 Mapeamento Semântico para a API

A Tabela 1 detalha o mapeamento semântico implementado no modelo Pydantic (`server/models.py`), convertendo colunas em inglês com caracteres especiais para tipos fortemente tipados.

Tabela 1: Mapeamento dos Atributos do CSV para o Esquema da API e Banco de Dados

Coluna Original (CSV)	Campo na API/DB	Tipo	Descrição Semântica
`Patient Number`	`paciente_id`	`int`	Identificador unívoco do paciente
`Heart Rate (bpm)`	`frequencia_cardiaca`	`int`	Frequência cardíaca (bpm)
`SpO2 Level (%)`	`spo2`	`int`	Saturação de oxigênio periférico
`Systolic Blood Pressure`	`pressao_sistolica`	`int`	Pressão arterial sistólica (mmHg)
`Diastolic Blood Pressure`	`pressao_diastolica`	`int`	Pressão arterial diastólica (mmHg)
`Body Temperature (°C)`	`temperatura`	`float`	Temperatura corpórea (°C)
`Fall Detection`	`deteccao_queda`	`str`	Sensor de queda (Yes/No)
`Predicted Disease`	`doenca_prevista`	`str`	Diagnóstico clínico correlacionado
`Data Accuracy (%)`	`precisao_dados`	`int`	Índice de confiabilidade do sensor
`Heart Rate Alert`	`alerta_freq_cardiaca`	`str`	Alerta de FC (NORMAL/ABNORMAL)
`SpO2 Level Alert`	`alerta_spo2`	`str`	Alerta clínico de hipóxia
`Blood Pressure Alert`	`alerta_pressao`	`str`	Alerta de hipertensão/hipotensão
`Temperature Alert`	`alerta_temperatura`	`str`	Alerta térmico (hipo/hipertermia)

4 Recapitulação dos Marcos 1 e 2

4.1 Marco 1: Servidor Base e Ingestão de Dados

No Marco 1, consolidou-se o fluxo unitário de transmissão e armazenamento de telemetria médica. O componente leitor (`client/csv_sender.py`) lê o dataset, converte as linhas para payloads JSON estruturados e os submete via requisições HTTP `POST /dados` a um servidor base FastAPI rodando na porta 8001. O servidor valida os dados através do schema Pydantic, injeta metadados (timestamp e identificador `server-01`) e persiste o registro no banco relacional local SQLite (`server1.db`), conforme esquematizado na Figura 2.



Figura 2: Arquitetura do Marco 1: Ingestão direta e persistência mononó.

4.2 Marco 2: Cluster com Balanceador de Carga Round Robin

No Marco 2, o sistema escalou para uma topologia com três instâncias idênticas e isoladas da aplicação executando nas portas 8001, 8002 e 8003. Para distribuir o tráfego de maneira

transparente, foi introduzido o Balanceador de Carga (`load_balancer/balancer.py`) na porta 8000, implementando o algoritmo Round Robin clássico sobre o conjunto $\mathcal{S} = \{s_0, s_1, s_2\}$:

$$\text{Servidor Alvo}(r) = s_{r \pmod 3} \tag{1}$$

A Figura 3 ilustra a topologia do Marco 2. Cada servidor gravava exclusivamente no seu respectivo banco SQLite local (`server1.db`, `server2.db` ou `server3.db`).

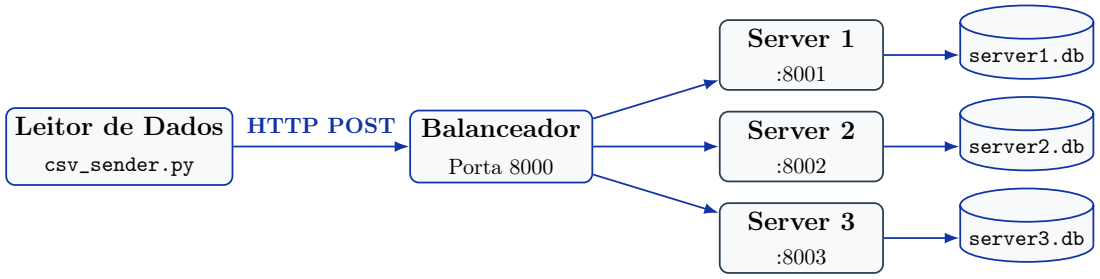


Figura 3: Topologia do Marco 2: Balanceamento Round Robin entre 3 nós independentes.

4.3 O Problema Identificado no Marco 2 (Motivação do Marco 3)

Embora o Marco 2 tenha resolvido a divisão homogênea da carga de escrita, ele revelou uma limitação crítica de consistência em sistemas distribuídos: **o isolamento estrito dos bancos de dados locais**.

Como cada servidor operava como uma ilha de dados, ao enviar 12 requisições, cada banco retinha apenas 4 registros intercalados. Consequentemente:

1. **Inconsistência de Leitura e Erros 404:** Uma requisição clínica `GET /dados/{paciente_id}` direcionada pelo balanceador para o Servidor 02 retornava `HTTP 404 Not Found` caso a medição daquele paciente tivesse sido processada pelo Servidor 01 ou 03.

2. **Falta de Desacoplamento Temporal:** O emissor dependia da disponibilidade síncrona dos servidores HTTP. Se o cluster estivesse temporariamente saturado, medições de pacientes eram descartadas.

3. **Rigidez Operacional:** O cluster não previa adição dinâmica de capacidade computacional sem reinicialização do balanceador.

Essas deficiências fundamentaram o escopo de inovações desenvolvidas no **Marco 3**.

5 Marco 3 — Middleware MOM, Replicação e Escalabilidade

O Marco 3 transforma a arquitetura em um sistema distribuído empresarial com suporte a replicação broadcast em tempo real, tolerância a picos de tráfego e escalabilidade elástica sob demanda.

5.1 Nova Topologia Integrada do Sistema Distribuído

A Figura 4 apresenta a topologia arquitetural completa implementada no Marco 3. O pipeline divide-se em: camada produtora, middleware de mensageria AMQP, consumidores concorrentes de replicação, cluster quádruplo de servidores e balanceador com gestão dinâmica de nós.

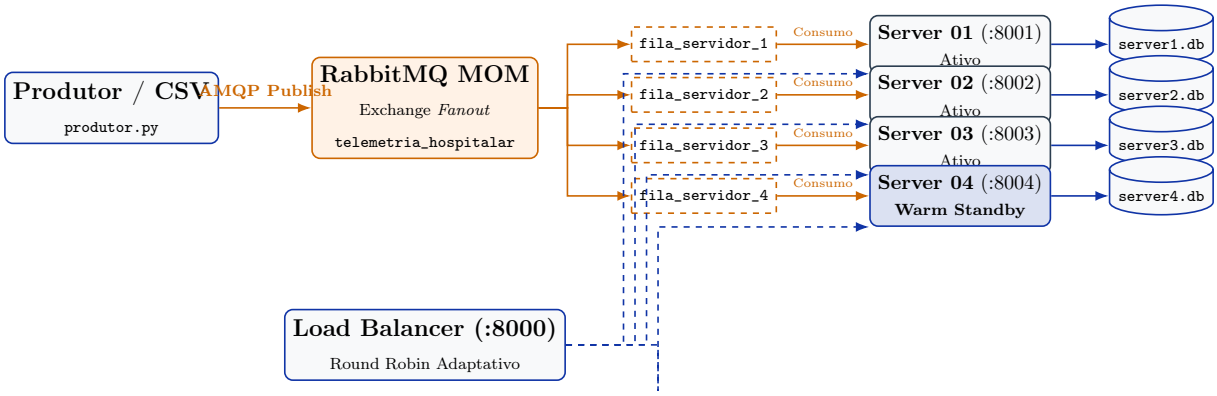


Figura 4: Topologia do Marco 3: Middleware RabbitMQ Fanout, Replicação nos 4 bancos e 4º Nó em Warm Standby.

5.2 Middleware Orientado a Mensagens (RabbitMQ)

No contexto de Sistemas Distribuídos, o middleware MOM (*Message-Oriented Middleware*) atua como o mediador desacoplador entre produtores e consumidores. Seus benefícios fundamentais comprovados no projeto são:

- **Desacoplamento Espacial:** O produtor desconhece os endereços IP, portas ou o quantitativo de servidores em execução.
- **Desacoplamento Temporal:** As mensagens são persistidas no broker. Caso os servidores estejam temporariamente desligados ou processando lotes pesados, as telemetrias aguardam nas filas sem perda de pacotes.
- **Confiabilidade e Garantia de Entrega:** Configuração de mensagens persistentes (`delivery_mode=2`) combinada com confirmações explícitas de processamento (`basic_ack` e `basic_nack` com `requeue=True`).

O módulo `middleware/config.py` centraliza a topologia AMQP:

```
EXCHANGE_TELEMETRIA = "telemetria_hospitalar"
TIPO_EXCHANGE = "fanout"
FILAS_SERVIDORES = [
    "fila_servidor_1",
    "fila_servidor_2",
    "fila_servidor_3",
    "fila_servidor_4",
]
```

Listing 1: Definição da Infraestrutura AMQP em middleware/config.py

5.3 Replicação Ativa Broadcast (Padrão Fanout)

Para solucionar a inconsistência de dados do Marco 2, adotamos o padrão de **Replicação Ativa via Exchange Fanout**. Quando o produtor publica uma medição na exchange `telemetria_hospitalar`, o RabbitMQ clona e entrega instantaneamente a mensagem para **todas as quatro filas** simultaneamente.

O script `middleware/consumidor.py` foi implementado com modo multithread (`-multi`), instanciando quatro consumidores paralelos independentes. Cada consumidor drena sua respectiva fila e persiste a medição no banco SQLite correspondente:

- Consumidor 1 → `fila_servidor_1` → `server1.db`
- Consumidor 2 → `fila_servidor_2` → `server2.db`
- Consumidor 3 → `fila_servidor_3` → `server3.db`
- Consumidor 4 → `fila_servidor_4` → `server4.db`

Impacto na Consistência: Ao final do envio de K medições, todos os quatro bancos contêm rigorosamente os mesmos K registros. Assim, qualquer consulta `GET /dados/{paciente_id}` direcionada pelo balanceador para qualquer servidor resulta em `HTTP 200 OK` imediato com o histórico clínico completo.

5.4 Escalabilidade Horizontal Elástica (Scale-Out e Scale-In)

Uma das inovações do Marco 3 é o suporte à elasticidade dinâmica de recursos computacionais:

- **O Conceito de Warm Standby:** O Servidor 04 (:8004) opera continuamente sincronizado em segundo plano, recebendo as réplicas do middleware no banco `server4.db`. Porém, no estado inicial, ele não recebe tráfego HTTP direto de clientes.
- **Scale-Out Dinâmico (Ativar Reserva):** Quando o cluster sofre picos de carga, o operador aciona a rota `POST /servidores/ativar-reserva` no balanceador. Instantaneamente, o balanceador promove o Servidor 04 para o anel ativo. Como o banco `server4.db` já estava replicado em tempo real, o nó entra em operação imediatamente, sem latência de sincronização inicial. O algoritmo Round Robin recalcula automaticamente o anel para $N = 4$ servidores:

$$\text{Servidor Alvo}(r) = s_{r \pmod 4} \tag{2}$$

  A carga unitária por servidor é reduzida de 33,3% para 25,0%.
- **Scale-In Dinâmico (Desativar Reserva):** Cessada a sobrecarga, a rota `POST /servidores/desativar-reserva` remove o Servidor 04 do anel ativo de roteamento HTTP, retornando com transparência e zero downtime para $N = 3$ servidores.

5.5 Visualização em Tempo Real e Acesso Remoto Seguro

Para demonstrações acadêmicas e validação operacional, o Marco 3 integrou:

- **Monitor Web Interativo (:5000):** Interface moderna (`visualizacao/monitor.py`) provida de animações SVG dos pacotes trafegando entre nós, métricas de distribuição em tempo real e console de eventos.
- **Túnel Público Seguro via Cloudflare (`tunel.sh`):** Integração de túnel reverso criptografado HTTPS (`trycloudflare.com`), permitindo que bancas avaliadoras e dispositivos móveis externos (4G/5G) acessem a interface diretamente do PC do desenvolvedor sem necessidade de port forwarding ou abertura de portas em roteadores.
- **Mecanismo de Smart Polling Fallback:** Solução de engenharia desenvolvida para superar o buffering agressivo que CDNs e operadoras móveis aplicam em conexões HTTP streaming (Server-Sent Events). Se o stream SSE for retido, a interface chaveia em 350ms para o endpoint `/api/eventos-poll`, garantindo animação contínua e sem travamentos.

6 Decisões de Engenharia e Desafios Superados no Marco 3

Durante o desenvolvimento do Marco 3, superamos desafios técnicos fundamentais:

1. **Transição de Fila Ponto-a-Ponto para Fanout Broadcast:** Inicialmente, a arquitetura utilizava uma única fila compartilhada concorrente. Contudo, filas ponto-a-ponto distribuem mensagens em revezamento, perpetuando o particionamento de dados. A migração para a exchange *Fanout* com 4 filas dedicadas resolveu definitivamente o problema da replicação de dados.
2. **Tratamento de Concorrência e Bloqueios no SQLite:** A persistência simultânea em alta taxa expôs erros de travamento (`database is locked`). Superamos essa limitação ativando o modo **WAL (Write-Ahead Logging)**, configurando `timeout=30.0s` em todas as conexões SQLite e assegurando que cada thread de consumo AMQP utilize sua própria conexão isolada.
3. **Contorno do Buffering de Streaming de Proxies/Cloudflare:** Ao expor o monitor via túnel Cloudflare, constatou-se que o stream SSE ficava retido no buffer da CDN. Implementamos duas contramedidas: injeção de *padding* de 4KB na inicialização da resposta SSE com cabeçalhos `no-transform`, e a criação do endpoint `/api/eventos-poll` com buffer circular de eventos recentes, ativado automaticamente pela interface como fallback transparente.
4. **Resolução de Endereçamento Loopback (IPv4 vs. IPv6):** Em ambientes Linux com pilha dupla, utilitários de túnel tentavam conectar-se ao backend via IPv6 `[::1]:5000`, resultando em recusas de conexão (`connection refused`). Padronizamos todas as ligações de túnel e scripts para o endereço IPv4 literal `127.0.0.1`.

7 Validação e Resultados Experimentais

A validação do projeto conta com uma suíte abrangente de **51 testes automatizados** com `pytest`. A Tabela 2 sumariza a cobertura dos testes.

Tabela 2: Sumário Completo da Suíte de Testes Automatizados (51 Testes)

Módulo de Teste	**Qtd.**	**Objetivo da Validação**
`tests/test_marco1.py`	17	Integridade do CSV, parsing, schema Pydantic, inserções SQLite, paginação (`limit/offset`) e consultas individuais.
`tests/test_marco2.py`	9	Aritmética modular Round Robin, inicialização simultânea dos 3 servidores e roteamento HTTP transparente.
`tests/test_marco3.py`	7	Replicação Fanout nos 4 bancos, eliminação de erros 404, ativação do servidor reserva (:8004), divisão uniforme com 4 nós e desativação elástica.
`tests/test_middleware.py`	11	Conectividade RabbitMQ, declaração da exchange Fanout, persistência AMQP, confirmações ACK/NACK com requeue e ordenação FIFO.
`tests/test_visualizacao.py`	7	Servidor monitor FastAPI, entrega da interface HTML, rotas de controle de simulação, SSE e integridade do teste de carga.
Total de Testes Aprovados	**51/51**	**Taxa de Sucesso: 100% (tempo de execução: 5,44s)**

7.1 Comprovação Experimental da Divisão de Carga

Submeteu-se o sistema a um lote de 120 requisições consecutivas para aferição da equidade de balanceamento em ambos os regimes de operação:

Tabela 3: Distribuição de Carga Experimental em Regime Normal e Scale-Out

Cenário	**Server 01 (:8001)**	**Server 02 (:8002)**	**Server 03 (:8003)**	**Server 04 (:800**
Normal ($N = 3$)	40 (33,3%)	40 (33,3%)	40 (33,3%)	0 (Standby)
Scale-Out ($N = 4$)	30 (25,0%)	30 (25,0%)	30 (25,0%)	30 (25,0%)

Os dados experimentais confirmam a exata convergência do algoritmo Round Robin elástico.

8 Roteiro Prático de Demonstração e Verificação (Marco 3)

8.1 Inicialização Integrada do Cluster

Todo o ecossistema (RabbitMQ, 4 servidores, balanceador, consumidores Fanout e monitor) pode ser inicializado com um único comando:

```
cd "Projeto SD"
# Modo padrao com Middleware RabbitMQ ativo:
./executar.sh

# Ou inicializacao com criacao de Tunel Publico Seguro (Cloudflare HTTPS):
./executar.sh --tunel
```

8.2 Envio de Telemetria via Middleware AMQP

Para publicar medições reais através da exchange Fanout do RabbitMQ:

```
# Envio de lote com 20 registros a cada 0.3 segundos:
python client/csv_sender.py --via-rabbitmq --intervalo 0.3 --limite 20
```

8.3 Demonstração da Escalabilidade Elástica

Para testar a adição dinâmica de capacidade (Scale-Out) e posterior redução (Scale-In):

```
# 1. Ativar Servidor 04 Reserva (Scale-Out: N=3 -> N=4):
curl -X POST http://localhost:8000/servidores/ativar-reserva

# 2. Enviar carga de teste e verificar a distribuicao nos 4 servidores:
python client/csv_sender.py --url http://localhost:8000/dados --intervalo 0.1 --limite 16

# 3. Desativar Servidor 04 Reserva (Scale-In: N=4 -> N=3):
curl -X POST http://localhost:8000/servidores/desativar-reserva
```

8.4 Evidência da Replicação Consistente nos 4 Bancos

Após o envio de 20 medições via RabbitMQ, execute no terminal para auditar os quatro bancos SQLite:

```
for i in 1 2 3 4; do
    echo -n "Server $i (databases/server$i.db): "
    sqlite3 databases/server$i.db "SELECT COUNT(*) FROM dados_pacientes;"
done
```

Resultado Obtido:

```
Server 1 (databases/server1.db): 20
Server 2 (databases/server2.db): 20
Server 3 (databases/server3.db): 20
Server 4 (databases/server4.db): 20
```

O resultado comprova que a replicação broadcast Fanout eliminou o problema do particionamento, mantendo sincronia estrita entre todas as réplicas.

9 Conclusão e Próximos Passos

A conclusão do **Marco 3** marca a consolidação do projeto como um sistema distribuído robusto, consistente e elástico. A introdução do middleware RabbitMQ proporcionou o desacoplamento necessário para ingestão assíncrona de telemetria médica em alta escala, enquanto a replicação broadcast Fanout assegurou a integridade dos dados clínicos em múltiplos bancos relacionais.

Além disso, a implementação do mecanismo de escalabilidade horizontal (*Scale-Out/Scale-In*) com o quarto servidor em *Warm Standby* e o suporte a acesso remoto seguro demonstram a maturidade de engenharia da solução.

Próximos Passos (Marco 4 — Tolerância a Falhas Avançada):

- Implementação de *Health Checks* ativos e passivos no Load Balancer para detecção instantânea de queda de nós.
- Mecanismo automático de *Failover*: redirecionamento transparente de requisições em falha para servidores saudáveis sem devolução de erro 500 ao cliente.
- Fila de retenção no middleware para armazenamento de requisições caso múltiplos nós fiquem offline, com descarga automática assim que o serviço for restabelecido.